

Celery Async Task Queue

Master background tasks, job scheduling, and async processing in Django — from basic setup to advanced patterns used in real production systems.

What is Celery

Installation

Basic Tasks

Celery Beat

Chains & Groups

Retry Logic

Monitoring

Advanced Patterns

With Docker



What is Celery and Why Do We Need It?

Celery is a **distributed task queue** for Python. It lets you run tasks asynchronously — meaning in the background — without blocking the main web request.

🔴 PROBLEM — WITHOUT CELERY

User submits a registration form → Your Django view sends a welcome email (takes 3-5 seconds) → User waits staring at loading screen → Bad user experience. Now imagine sending 10,000 emails or generating a PDF report — users would wait forever.

🟢 SOLUTION — WITH CELERY

User submits form → Django immediately returns "Registration successful!" → Celery sends the email in the background → User never waits. The heavy work happens invisibly.

🔗 Real-World Use Cases



Email / SMS Sending

Welcome emails, OTPs, order confirmations, newsletters



Report Generation

PDF/Excel exports, analytics reports, data exports



Image Processing

Resize thumbnails, compress uploads, generate previews



Push Notifications

Firebase, APNs notifications to mobile apps



Scheduled Tasks

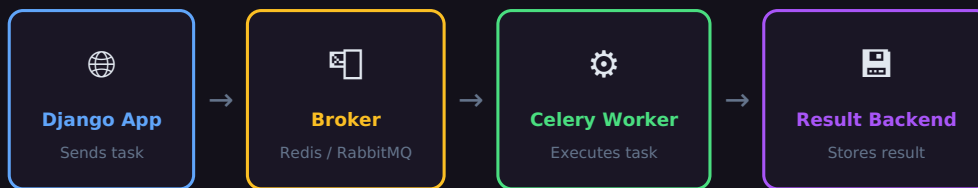
Daily backups, weekly digests, auto-reminders



API Calls

Third-party API calls, webhooks, data sync

🔧 How Celery Works — The Architecture



1 Django sends a task

Your view calls `send_email.delay(user_id)` — this immediately returns, Django does NOT wait.

2 Task goes to Broker (Redis)

Redis acts as a message queue. Think of it as a post office — tasks are "letters" waiting to be processed.

3 Celery Worker picks up task

A separate process (the worker) constantly watches Redis and picks up new tasks to execute.

4 Result stored in backend

After execution, the result (success/failure/return value) is saved in Redis for later retrieval.



Installation & Configuration

BASIC

📦 Install Packages

BASH

```
# Install Celery with Redis support
pip install celery redis django-celery-results django-celery-beat

# Add to requirements.txt
celery==5.3.6
redis==5.0.1
django-celery-results==2.5.1
django-celery-beat==2.6.0
```

⚙️ Django Settings

```

INSTALLED_APPS = [
    # ... your existing apps ...
    'django_celery_results', # Store task results in DB
    'django_celery_beat',    # Schedule tasks
]

# === CELERY CONFIGURATION ===

# Broker — Redis is the "message queue"
CELERY_BROKER_URL = 'redis://localhost:6379/0'

# Result Backend — where to store task results
CELERY_RESULT_BACKEND = 'django-db'

# Cache backend (optional but recommended)
CELERY_CACHE_BACKEND = 'django-cache'

# Timezone (important for scheduled tasks)
CELERY_TIMEZONE = 'Asia/Karachi'

# Serialize task arguments as JSON (safer than pickle)
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_ACCEPT_CONTENT = ['json']

```

🔗 celery.py — Celery Application Setup

Create this file inside your **main project folder** (same level as settings.py):

```

import os
from celery import Celery

# Tell Celery which Django settings to use
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myproject.settings')

# Create the Celery application instance
app = Celery('myproject')

# Load config from Django settings — all CELERY_ prefixed keys
app.config_from_object('django.conf:settings', namespace='CELERY')

# Auto-discover tasks.py in all installed apps
app.autodiscover_tasks()

```

🔗 __init__.py — Load Celery with Django

```

# This ensures Celery is loaded when Django starts
from .celery import app as celery_app

__all__ = ('celery_app',)

```

▶ Starting Celery Worker

```

# Make sure Redis is running first!

```

```
redis-server

# Start Celery worker (in a separate terminal)
celery -A myproject worker --loglevel=info

# -A myproject = use your project's celery app
# --loglevel=info = show info logs

# Windows users (use eventlet or solo pool)
celery -A myproject worker --loglevel=info --pool=solo
```



Creating and Calling Tasks

BASIC

Creating a Task

Create a `tasks.py` file inside any Django app. Use the `@shared_task` decorator:

MYAPP/TASKS.PY

```
from celery import shared_task
from django.core.mail import send_mail
from django.contrib.auth.models import User
import time

# @shared_task works without importing the Celery app directly
# This is the recommended approach for reusable apps
@shared_task
def send_welcome_email(user_id):
    """Send welcome email to new user"""
    # Always fetch objects inside the task, not pass full objects
    user = User.objects.get(id=user_id)

    send_mail(
        subject=f'Welcome, {user.first_name}!',
        message='Thanks for registering on our platform.',
        from_email='noreply@example.com',
        recipient_list=[user.email],
    )
    return f'Email sent to {user.email}'

@shared_task
def generate_report(report_id):
    """Simulate a slow report generation"""
    time.sleep(10) # Simulating heavy work
    return f'Report {report_id} generated successfully'

@shared_task
def add_numbers(x, y):
    """Simple task for testing"""
    return x + y
```

⚠ IMPORTANT — PASS IDS, NOT OBJECTS

Never pass a Django model instance to a task (e.g., don't pass `user`, pass `user.id`). Objects can't be serialized to JSON. Always pass simple values like integers, strings, and fetch the object inside the task.

🔗 Three Ways to Call a Task

VIEWS.PY

```

from .tasks import send_welcome_email, add_numbers
from rest_framework.response import Response
from rest_framework.decorators import api_view

@api_view(['POST'])
def register_user(request):
    # ... create user logic ...
    user = User.objects.create_user(**request.data)

    # METHOD 1: .delay() — Simplest way, just pass args
    send_welcome_email.delay(user.id)

    # METHOD 2: .apply_async() — More control
    send_welcome_email.apply_async(
        args=[user.id],
        countdown=60,      # Wait 60 seconds before executing
        expires=3600,      # Task expires after 1 hour if not run
    )

    # METHOD 3: .apply_async() with eta (specific time)
    from datetime import datetime, timedelta
    eta = datetime.now() + timedelta(hours=1)
    send_welcome_email.apply_async(args=[user.id], eta=eta)

    return Response({'message': 'Registered! Welcome email coming soon.'})

```

Getting Task Results

```

from .tasks import add_numbers

# Send task and get a result object
result = add_numbers.delay(10, 20)

# Get the task ID (unique identifier)
task_id = result.id      # e.g., "550e8400-e29b-41d4-a716-446655440000"

# Check task status
print(result.status)     # PENDING, STARTED, SUCCESS, FAILURE, RETRY

# Get result (WARNING: this BLOCKS until task completes)
value = result.get(timeout=10)  # Returns 30

# Check if done without blocking
if result.ready():
    print(result.result)    # The return value

# Check if failed
if result.failed():
    print(result.traceback)

```



Celery Beat — Scheduled Tasks

INTERMEDIATE

Celery Beat is a scheduler that sends tasks at regular intervals — like a cron job but integrated with Django. Use it for daily reports, cleanup jobs, nightly backups, etc.

Settings Configuration

```

from celery.schedules import crontab

```

```

CELERY_BEAT_SCHEDULE = {

    # Run every 30 minutes
    'send-reminder-every-30-minutes': {
        'task': 'myapp.tasks.send_reminders',
        'schedule': 1800, # seconds
    },

    # Run every day at 8:00 AM
    'daily-report-at-8am': {
        'task': 'myapp.tasks.generate_daily_report',
        'schedule': crontab(hour=8, minute=0),
    },

    # Run every Monday at 9 AM
    'weekly-email-monday': {
        'task': 'myapp.tasks.send_weekly_newsletter',
        'schedule': crontab(hour=9, minute=0, day_of_week=1),
    },

    # Run on 1st of every month
    'monthly-invoice': {
        'task': 'myapp.tasks.generate_invoices',
        'schedule': crontab(hour=0, minute=0, day_of_month=1),
    },

}

```

► Starting Celery Beat



BASH

```

# Terminal 1: Start Redis
redis-server

# Terminal 2: Start Celery Worker
celery -A myproject worker --loglevel=info

# Terminal 3: Start Celery Beat (the scheduler)
celery -A myproject beat --loglevel=info --scheduler django_celery_beat.schedulers:DatabaseScheduler

```



DATABASE SCHEDULER ADVANTAGE

With DatabaseScheduler, you can manage schedules from Django Admin — create, edit, disable schedules at runtime without restarting the server. Very useful in production.



Chains, Groups & Chords

INTERMEDIATE

Celery lets you compose multiple tasks together into powerful workflows using **primitives**.

⌘ Chain — Run Tasks in Sequence

Each task's output becomes the next task's input:



PYTHON

```

from celery import chain
from .tasks import validate_order, charge_payment, send_confirmation

# Real world: Process an order step by step
# Step 1: validate → Step 2: charge → Step 3: notify
order_pipeline = chain(

```

```

    validate_order.s(order_id),
    charge_payment.s(),          # receives output from validate_order
    send_confirmation.s()        # receives output from charge_payment
)

# .s() = signature — a lazy version of .delay()
result = order_pipeline.apply_async()

```

👥 Group — Run Tasks in Parallel

All tasks run at the same time, independently:



PYTHON

```

from celery import group
from .tasks import send_notification

# Send notification to 1000 users — all at the same time
user_ids = [1, 2, 3, 4, 5]

job = group(send_notification.s(user_id) for user_id in user_ids)
result = job.apply_async()

# Wait for all tasks to finish
results = result.get() # Returns list of results

```

🎵 Chord — Parallel + Callback

Run a group in parallel, then call a callback task when ALL are done:



PYTHON

```

from celery import chord
from .tasks import process_image, combine_results

# Process 5 images in parallel, then combine all results
image_ids = [101, 102, 103, 104, 105]

result = chord(
    process_image.s(img_id) for img_id in image_ids
)(combine_results.s()) # Called when ALL images are processed

```



Retry Logic & Error Handling

INTERMEDIATE

Real APIs fail. Networks drop. Databases go down momentarily. Celery has built-in retry mechanisms to handle these failures gracefully.



TASKS.PY

```

from celery import shared_task
from celery.exceptions import MaxRetriesExceededError
import requests
import logging

logger = logging.getLogger(__name__)

@shared_task(
    bind=True,          # Gives access to 'self' (the task instance)
    max_retries=3,       # Max 3 retry attempts
    default_retry_delay=60, # Wait 60 seconds between retries
)
def call_payment_api(self, order_id, amount):

```

```

"""
Call payment API with automatic retry on failure.
Will retry up to 3 times with 60 second delay.
"""
try:
    response = requests.post(
        'https://api.payment.com/charge',
        json={'order_id': order_id, 'amount': amount},
        timeout=30
    )
    response.raise_for_status() # Raises exception for 4xx, 5xx
    return response.json()

except requests.exceptions.Timeout as exc:
    logger.warning(f'Payment API timeout for order {order_id}, retrying...')
    # Retry with exponential backoff: 60s, 120s, 240s
    raise self.retry(exc=exc, countdown=60 * (2 ** self.request.retries))

except requests.exceptions.RequestException as exc:
    try:
        raise self.retry(exc=exc)
    except MaxRetriesExceededError:
        # All retries exhausted - handle gracefully
        logger.error(f'Payment failed for order {order_id} after all retries')
        # Notify admin, update order status to failed, etc.
        raise # Re-raise so Celery marks task as FAILURE

```

🔗 Task Hooks — on_success, on_failure

TASKS.PY

```

from celery import Task

class CallbackTask(Task):
    def on_success(self, retval, task_id, args, kwargs):
        """Called when task succeeds"""
        logger.info(f'Task {task_id} completed: {retval}')

    def on_failure(self, exc, task_id, args, kwargs, einfo):
        """Called when task fails"""
        logger.error(f'Task {task_id} failed: {exc}')
        # Send alert to Slack, email to admin, etc.

@shared_task(base=CallbackTask)
def important_task(data):
    # task code here
    pass

```



Monitoring with Flower

INTERMEDIATE

Flower is a real-time web-based monitor for Celery. You can see active tasks, worker status, task history, and failed tasks — all from a dashboard.

BASH

```

# Install Flower
pip install flower

# Start Flower (run alongside your worker)
celery -A myproject flower --port=5555

# Now open: http://localhost:5555
# You'll see dashboard with workers, tasks, queues

```


📖 WHAT YOU CAN SEE IN FLOWER

Active workers and their status, tasks currently running, task history with success/failure counts, detailed info for each task (arguments, result, timing), ability to revoke (cancel) pending tasks, and worker resource usage.



Advanced Patterns

ADVANCED

🔗 Multiple Queues — Priority Routing

Separate urgent tasks (OTP SMS) from slow tasks (report generation) into different queues with different worker counts.



SETTINGS.PY + TASKS.PY

```
# settings.py — Define task routing
CELERY_TASK_ROUTES = {
    'myapp.tasks.send_otp': {'queue': 'high_priority'},
    'myapp.tasks.send_newsletter': {'queue': 'low_priority'},
    'myapp.tasks.generate_report': {'queue': 'reports'},
}

# Start separate workers for each queue
# High priority: 4 workers, concurrency 8
# celery -A myproject worker -Q high_priority --concurrency=8
# Low priority: 1 worker
# celery -A myproject worker -Q low_priority --concurrency=2

# tasks.py — Assign task to specific queue
@shared_task(queue='high_priority')
def send_otp(user_id, otp):
    pass

# Or override at call time
send_newsletter.apply_async(args=[campaign_id], queue='low_priority')
```

🔗 Task Progress Updates (via WebSocket)

Update the frontend on task progress in real-time using Redis as a communication channel:



TASKS.PY

```
from celery import shared_task
from channels.layers import get_channel_layer
from asgiref.sync import async_to_sync

@shared_task(bind=True)
def process_large_file(self, file_path, user_id):
    channel_layer = get_channel_layer()
    total_steps = 100

    for step in range(total_steps):
        # Do actual work...
        time.sleep(0.1)

        # Update task state (visible in Celery results)
        self.update_state(
            state='PROGRESS',
            meta={'current': step, 'total': total_steps}
        )

        # Push update to WebSocket (Django Channels)
        async_to_sync(channel_layer.group_send)(
            f'user_{user_id}',
            {'type': 'task_progress', 'progress': step + 1}
        )
```

```
)

return {'status': 'done', 'file': file_path}
```

🔒 Preventing Duplicate Tasks (Task Locking)

Ensure a task doesn't run twice simultaneously (e.g., don't charge the same user twice):

TASKS.PY

```
from django.core.cache import cache

@shared_task
def process_payment(order_id):
    """Ensure this task only runs once per order"""
    lock_key = f'payment_lock_{order_id}'

    # Try to acquire lock (expires in 5 minutes)
    lock_acquired = cache.add(lock_key, 'locked', timeout=300)

    if not lock_acquired:
        # Another worker is already processing this order
        logger.warning(f'Order {order_id} already being processed')
        return 'Already processing'

    try:
        # Do the actual payment processing
        charge_user(order_id)
        return 'Payment successful'
    finally:
        # Always release the lock
        cache.delete(lock_key)
```

Celery + Django + Docker

ADVANCED

The complete production setup with Django, Celery Worker, Celery Beat, Redis, and Flower — all in Docker Compose:

DOCKER-COMPOSE.YML

```
version: '3.8'

services:

    # 1. Django Web App
    web:
        build: .
        command: python manage.py runserver 0.0.0.0:8000
        ports:
            - "8000:8000"
        env_file:
            - .env
        depends_on:
            - db
            - redis

    # 2. Celery Worker — executes tasks
    celery_worker:
        build: .
        command: celery -A myproject worker --loglevel=info
        env_file:
            - .env
        depends_on:
            - redis
            - db
```

```
# 3. Celery Beat — schedules periodic tasks
celery_beat:
  build: .
  command: celery -A myproject beat --loglevel=info --scheduler django_celery_beat.schedulers:DatabaseScheduler
  env_file:
  - .env
  depends_on:
  - redis
  - db

# 4. Flower — monitoring dashboard
flower:
  build: .
  command: celery -A myproject flower --port=5555
  ports:
  - "5555:5555"
  depends_on:
  - celery_worker

# 5. Redis — message broker
redis:
  image: redis:7-alpine

# 6. PostgreSQL
db:
  image: postgres:15
  env_file:
  - .env
  volumes:
  - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

🔧 START EVERYTHING

Run `docker-compose up --build` and you have Django + Celery Worker + Beat + Flower + Redis + PostgreSQL all running together. Access your app at `:8000` and Flower at `:5555`.

📋 Final Checklist for Interns

TASK	LEVEL	GOAL
Install Celery + Redis, setup <code>celery.py</code>	BASIC	Make it run
Create a task with <code>@shared_task</code> , call with <code>.delay()</code>	BASIC	First async task
Send welcome email async on user registration	BASIC	Real use case
Setup Celery Beat — daily cleanup job	INTERMEDIATE	Scheduled task
Add retry logic with exponential backoff	INTERMEDIATE	Production ready
Setup Flower and monitor tasks	INTERMEDIATE	Observability
Implement task chains for order processing	INTERMEDIATE	Workflows
Multiple queues with priority routing	ADVANCED	Scale

